

实验报告二

1. 程序实现的功能

本程序在前一阶段词法分析与语法分析的基础上，实现了 C++ 语言子集的**语义分析与类型检查**。整体流程为：先由 Flex 和 Bison 完成词法、语法分析并建立语法树；若输入中不存在词法和语法错误，则继续对语法树进行递归遍历，完成符号表维护、作用域管理、类型推导和语义错误检查。主程序在 `yyparse()` 后调用 `semantic_analyze(syntax_root)` 进入语义分析阶段，若无语义错误则输出语法树。

程序主要实现了以下功能：

(1) 多层作用域下的符号表管理。 程序使用哈希表保存当前可见符号，并使用作用域栈维护嵌套语句块。进入函数体或复合语句块时调用 `enter_scope()` 建立新作用域，退出时调用 `leave_scope()` 删除当前层定义的符号，从而支持“内层变量屏蔽外层变量、离开内层后外层重新可见”的语义要求。符号分为变量、函数、结构体三类，分别处理查找和重定义。

(2) 类型系统与类型检查。 程序定义了基本类型、数组类型、结构体类型和函数类型四类语义类型，并通过 `type_equal()` 完成类型等价判断。结构体采用**名等价**，匿名结构体会被分配唯一隐藏名；数组采用递归表示，从而支持“基类型与维数相同即认为匹配”的实验要求。

(3) 函数定义与调用检查。 程序在分析函数定义时记录函数返回类型、形参数量和形参类型；在分析调用表达式时，检查被调用对象是否是函数、实参数量是否匹配、各位置参数类型是否一致，因此能够发现函数未定义、对普通变量使用 `()`、以及函数参数不匹配等错误。

(4) 表达式、数组和结构体访问检查。 程序通过 `analyze_Exp()` 递归分析表达式，返回表达式类型及左值属性，从而支持赋值检查、算术运算检查、逻辑运算检查、数组下标检查和结构体域访问检查。例如：对非数组使用 `[]`、数组下标不是整数、对非结构体使用 `.`、访问不存在的结构体域等错误都可以被检测出来。

(5) 结构体内部字段检查。 结构体定义中的 `DefList` 会以单独模式处理，从而能够检测同一结构体中字段重定义以及字段初始化错误。

此外，程序还对**级联报错**做了抑制处理：当某个子表达式已经因根本性错误得到空类型后，上层表达式不再继续触发新的类型错误，这样可以减少冗余报错，使输出更接近实验样例。

2. 程序编译方式

直接使用 `Makefile` 编译。在项目根目录下执行：

```
make clean
make
```

编译成功后会生成可执行文件 `parser`，运行方式为：

```
./parser <input-file>
```

3. 程序亮点

第一，语义分析被设计为**独立于语法分析动作之外的一遍语法树遍历**。语法分析阶段只负责建树，语义分析阶段只负责符号表和类型检查，这样模块划分更清晰，也便于后续扩展。

第二，程序采用了**哈希表 + 作用域链表**的符号表结构。哈希表保证查找效率，作用域链表保证离开语句块时可以快速删除当前层定义的符号，从而较自然地支持实验要求中的嵌套作用域。

第三，程序对**空语法单元不建树**的情况做了专门处理。例如在复合语句 `CompSt` 中，当 `DefList` 为空时，语法树可能只包含 `LC -> StmtList -> RC`。程序通过检查节点名字而不是写死孩子下标的方式识别 `DefList` 和 `StmtList`，从而避免语义分析漏掉函数体中的语句。